Teoria dos Grafos

Bruno Pereira dos Santos

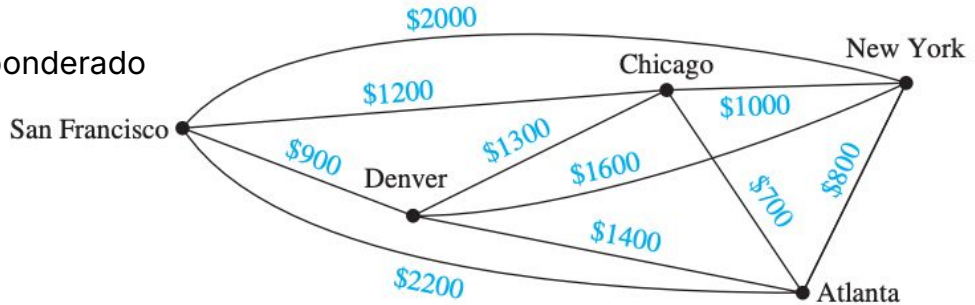
Aula baseada nos cursos de outros docentes da UFMG (Prof. Antonio Alfredo Ferreira Loureiro), USP (Prof. Paulo Feofiloff), UFBA, livros, artigos e outros.

09 - Minimum Spanning Trees

Árvores geradoras Mínimas

Introdução a Árvore Geradora Mínima

- Problema: Uma empresa deseja construir uma rede de comunicação conectando cinco centros de computadores.
  - Cada par de centros pode ser ligado por uma linha telefônica alugada
  - Cada ligação possui um custo mensal
  - Objetivo:
    - Garantir que todos os centros estejam conectados
    - Minimizar o custo total
- Modelagem
  - Representamos o problema com um grafo ponderado
  - Vértices → centros de computadores
  - Arestas → possíveis conexões
  - Pesos → custo mensal das ligações
- Solução
  - Encontrar uma árvore geradora que:
  - Conecte todos os vértices
  - Tenha o menor custo total possível



➜ Essa árvore é chamada de Árvore Geradora Mínima (Minimum Spanning Tree - MST)

Algoritmos para Árvores Geradoras Mínimas

- Muitos problemas podem ser resolvidos encontrando uma árvore geradora em um grafo ponderado tal que:
  - A soma dos pesos das arestas seja mínima
- Definição: Uma Árvore Geradora Mínima em um grafo conexo e ponderado é uma árvore geradora com a **menor soma possível dos pesos das arestas**
- Abordagem (Algoritmos Gulosos)
  - Construção passo a passo
  - Sempre escolhe a aresta de menor peso disponível
  - Respeita uma propriedade (não formar ciclos)
- Vamos ver 2 algoritmos:
  - Algoritmo de Prim
  - Algoritmo de Kruskal

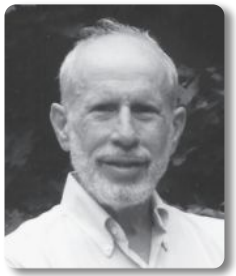
Algoritmos para Árvores Geradoras Mínimas (1)



ROBERT CLAY PRIM (NASCIDO EM 1921)

Robert Prim, nascido em Sweetwater, Texas, obteve seu bacharelado em engenharia elétrica em 1941 e seu Ph.D. em matemática pela Universidade de Princeton em 1949. Foi engenheiro na General Electric Company de 1941 a 1944, engenheiro e matemático no United States Naval Ordnance Lab de 1944 a 1949, e pesquisador associado na Universidade de Princeton de 1948 a 1949. Entre outros cargos que ocupou, destacam-se o de diretor de pesquisa em matemática e mecânica no Bell Telephone Laboratories de 1958 a 1961 e o de vice-presidente de pesquisa na Sandia Corporation. Atualmente, está aposentado.

O primeiro algoritmo que discutiremos foi originalmente descoberto pelo matemático tcheco Vojtěch Jarník em 1930, que o descreveu em um artigo em uma obscura revista tcheca. O algoritmo tornou-se bem conhecido quando foi redescoberto em 1957 por Robert Prim. Por isso, ele é conhecido como **algoritmo de Prim** (e às vezes como **algoritmo de Prim-Jarník**).



JOSEPH BERNARD KRUSKAL (1928–2010)

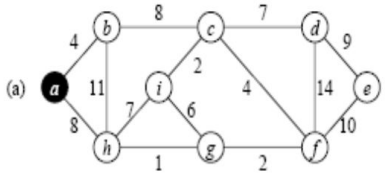
Joseph Kruskal nasceu na cidade de Nova York, onde seu pai era comerciante de peles e sua mãe promovia a arte do origami na televisão em seus primórdios. Kruskal estudou na Universidade de Chicago e obteve seu Ph.D. em Princeton em 1954. Foi instrutor de matemática em Princeton e na Universidade de Wisconsin, e posteriormente tornou-se professor assistente na Universidade de Michigan. Em 1959, passou a integrar a equipe técnica do Bell Laboratories, onde trabalhou até sua aposentadoria no final da década de 1990.

Kruskal descobriu seu algoritmo para gerar árvores geradoras mínimas quando ainda era um estudante de pós-graduação no segundo ano. Ele não tinha certeza se seu artigo de apenas duas páginas e meia sobre o tema valia a pena ser publicado, mas foi convencido por outros a submetê-lo. Seus interesses de pesquisa incluíam linguística estatística e psicometria. Além de seu trabalho com árvores geradoras mínimas, Kruskal também é conhecido por suas contribuições ao escalonamento multidimensional. Vale destacar que seus dois irmãos, Martin e William, também eram matemáticos renomados.

Algoritmo de Prim-Jarník

- Idéia básica:
  1. Comece escolhendo uma aresta de menor peso, adicione essa aresta à árvore geradora.
  2. A cada passo:
     - Escolha a **aresta de menor peso que seja incidente a um vértice já presente na árvore** E que não forme ciclo
     - Continue adicionando arestas até obter: n – 1 arestas

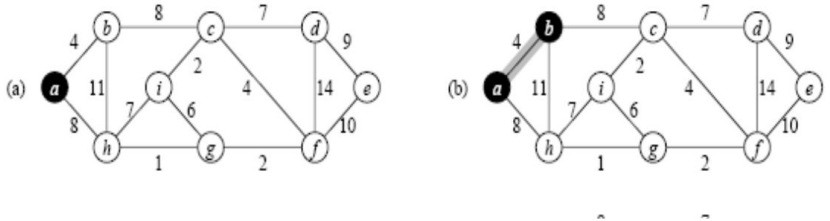
Algoritmo de Prim-Jarník (1)



- Tomando como vértice inicial A, crie uma fila de prioridades classificada pelos pesos das arestas conectando A.
- Repita o processo até que todos os vértices tenham sido visitados

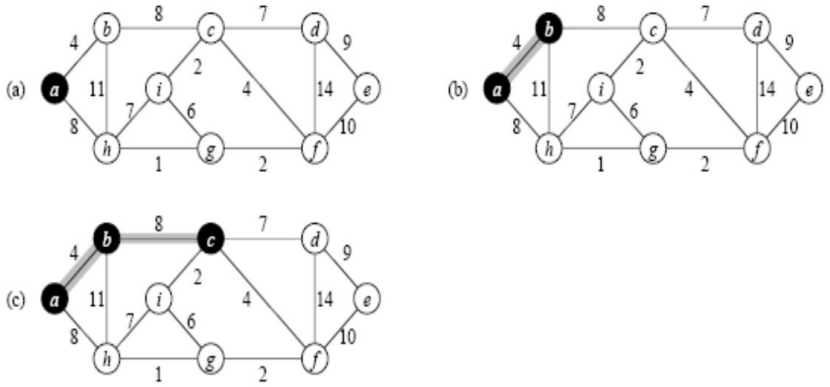
Algoritmo de Prim-Jarník (2)

- Tomando como vértice inicial A, crie uma fila de prioridades classificada pelos pesos das arestas conectando A.
- Repita o processo até que todos os vértices tenham sido visitados



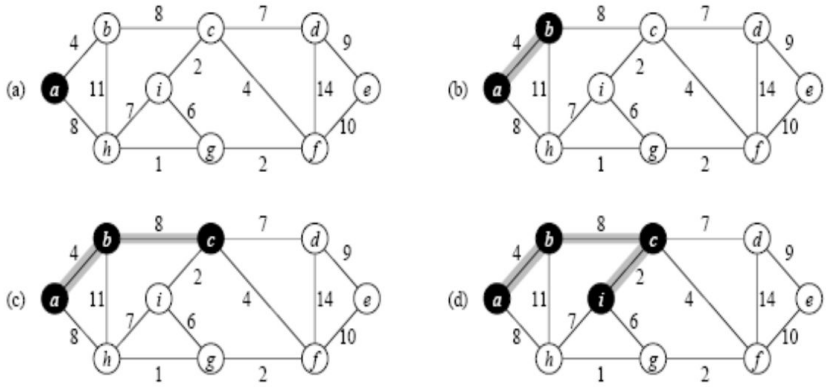
Algoritmo de Prim-Jarník (3)

- Tomando como vértice inicial A, crie uma fila de prioridades classificada pelos pesos das arestas conectando A.
- Repita o processo até que todos os vértices tenham sido visitados



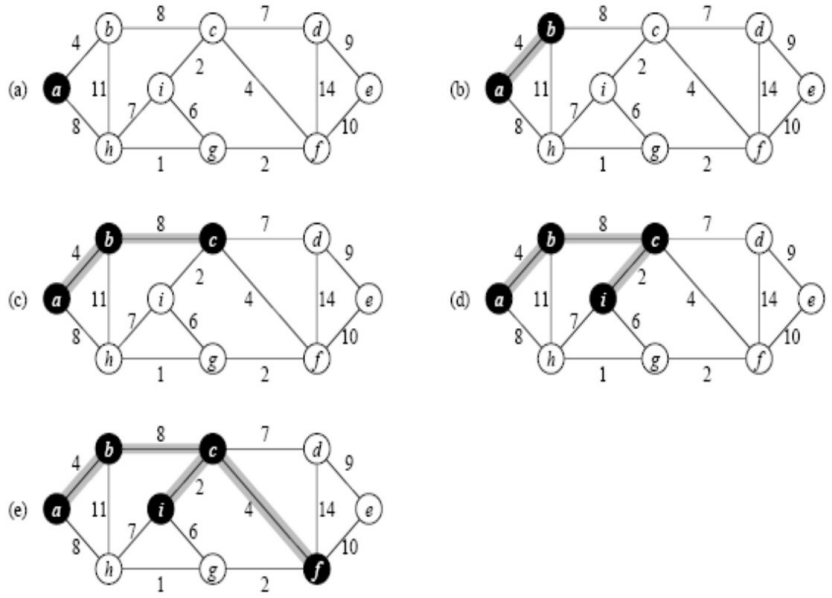
Algoritmo de Prim-Jarník (4)

- Tomando como vértice inicial A, crie uma fila de prioridades classificada pelos pesos das arestas conectando A.
- Repita o processo até que todos os vértices tenham sido visitados



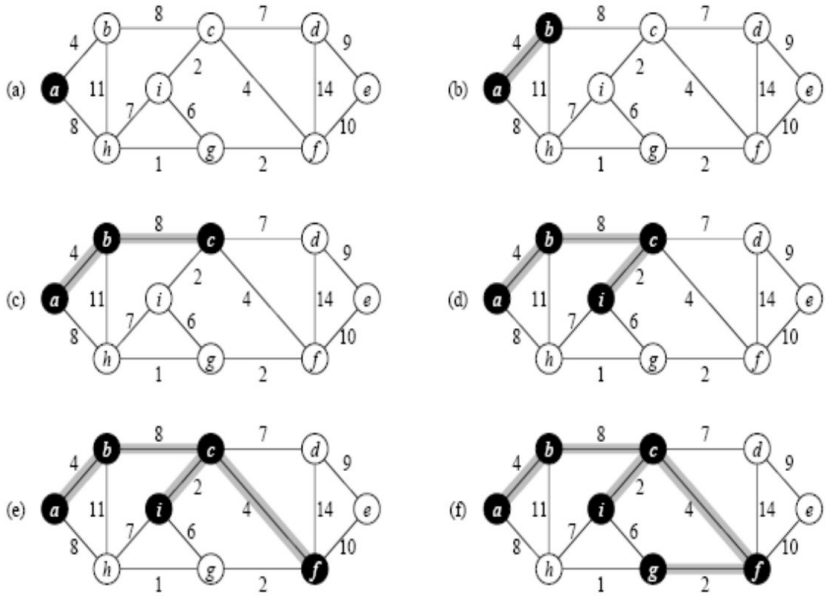
Algoritmo de Prim-Jarník (5)

- Tomando como vértice inicial A, crie uma fila de prioridades classificada pelos pesos das arestas conectando A.
- Repita o processo até que todos os vértices tenham sido visitados



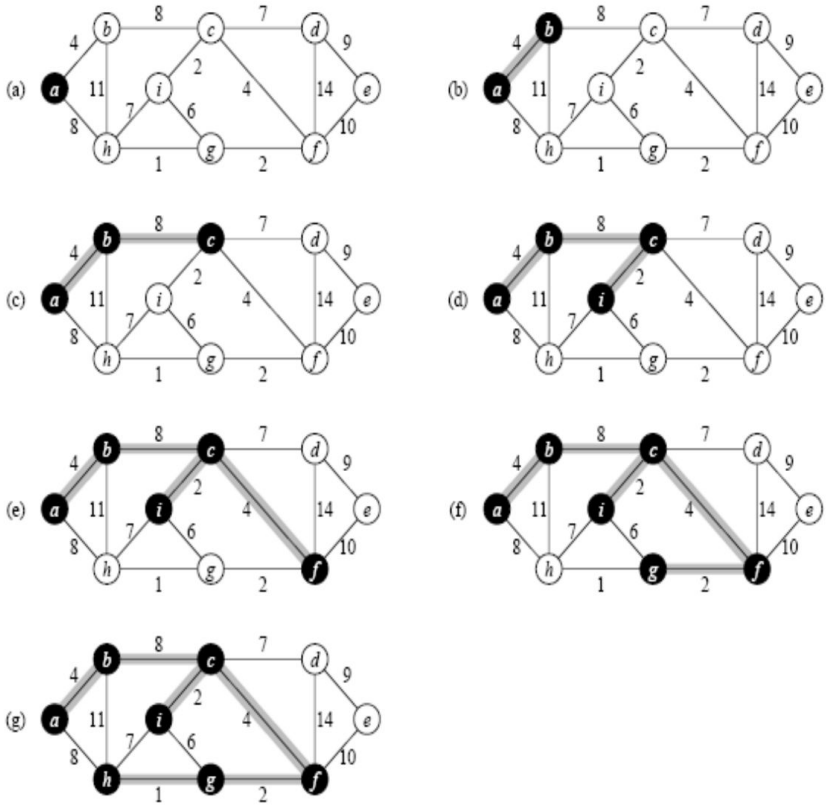
Algoritmo de Prim-Jarník (6)

- Tomando como vértice inicial A, crie uma fila de prioridades classificada pelos pesos das arestas conectando A.
- Repita o processo até que todos os vértices tenham sido visitados



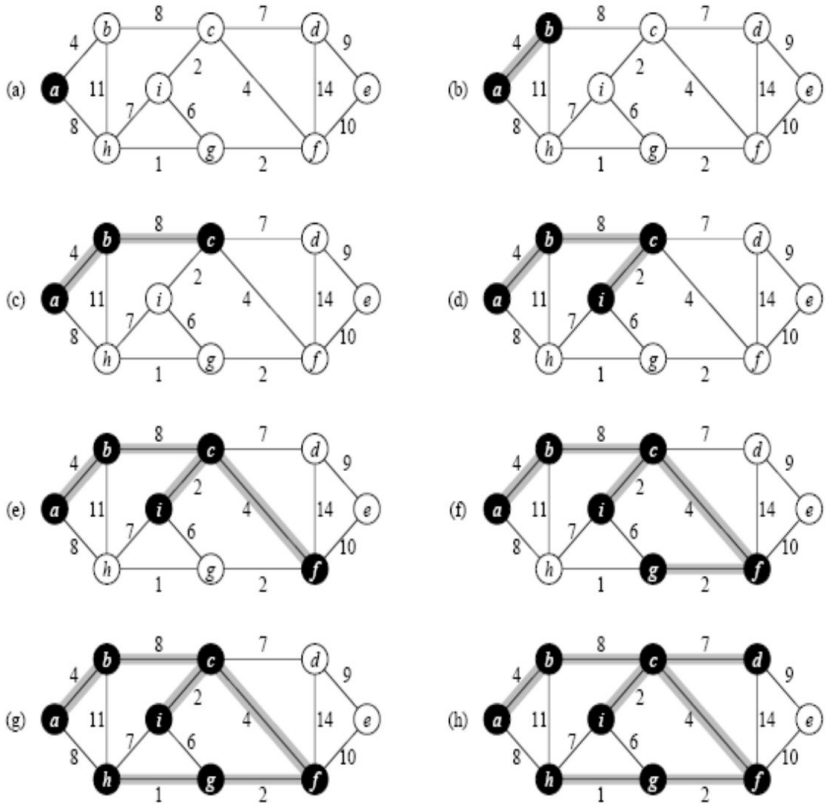
Algoritmo de Prim-Jarník (7)

- Tomando como vértice inicial A, crie uma fila de prioridades classificada pelos pesos das arestas conectando A.
- Repita o processo até que todos os vértices tenham sido visitados



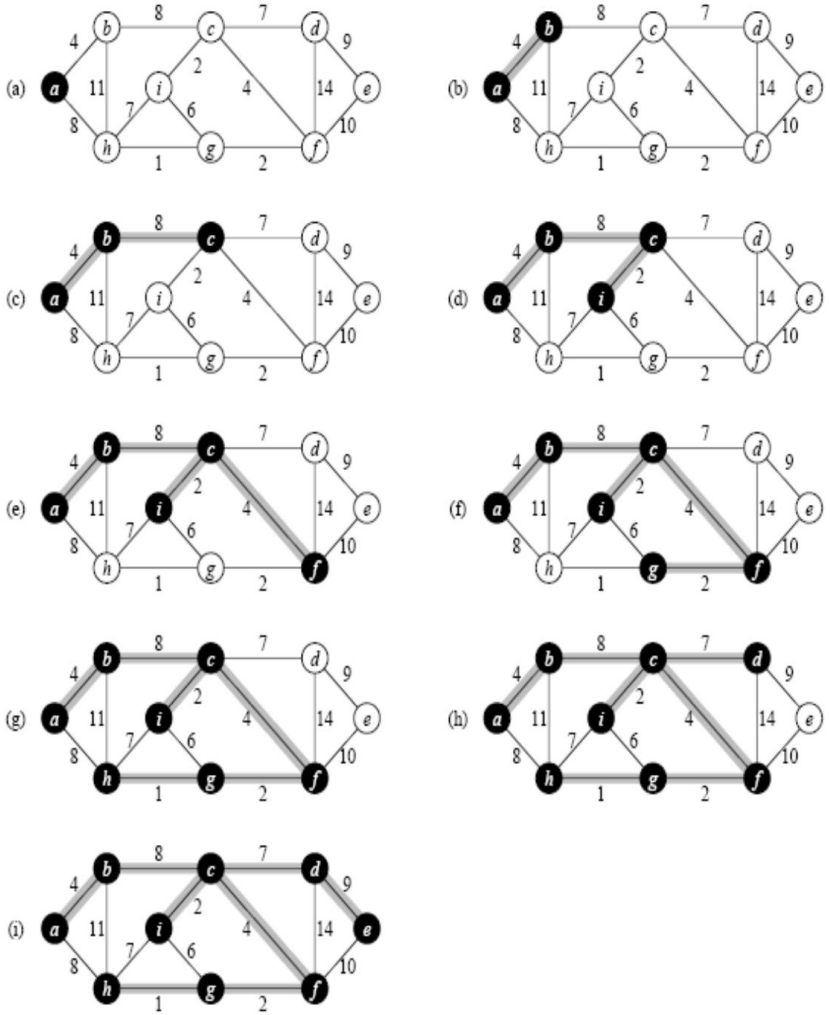
Algoritmo de Prim-Jarník (8)

- Tomando como vértice inicial A, crie uma fila de prioridades classificada pelos pesos das arestas conectando A.
- Repita o processo até que todos os vértices tenham sido visitados



Algoritmo de Prim-Jarník (9)

- Tomando como vértice inicial A, crie uma fila de prioridades classificada pelos pesos das arestas conectando A.
- Repita o processo até que todos os vértices tenham sido visitados



Algoritmo de Prim-Jarník (10)

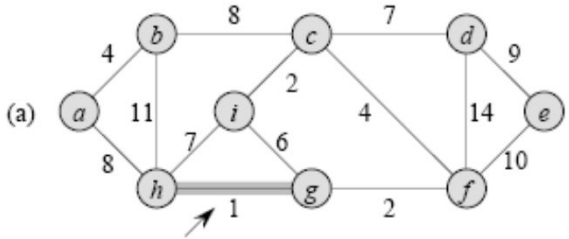
- **ALGORITHM 1 Prim's Algorithm.**

procedure *Prim*(G: weighted connected undirected graph with n vertices)
T := a minimum-weight edge
for $i := 1$ **to** $n - 2$
    e := an edge of minimum weight incident to a vertex in T and not forming a simple circuit in T if added to T
    $T := T$ with e added
return T {T is a minimum spanning tree of G}

Algoritmo de Kruskal

- Idéia básica:
  - Seleciona a aresta de menor peso que conecta duas árvores de uma floresta.
  - Repita o processo até que todos os vértices estejam conectados sempre preservando a invariante de se ter uma árvore.

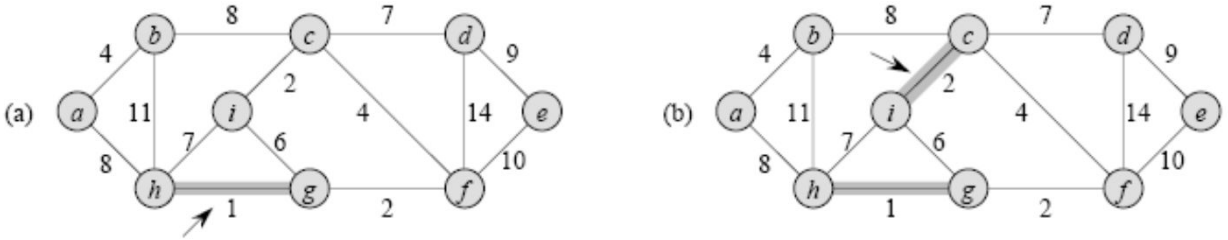
Algoritmo de Kruskal (1)



- Seleciona a aresta de menor peso que conecta duas árvores de uma floresta.
- Repita o processo até que todos os vértices estejam conectados sempre preservando a invariante de se ter uma árvore.

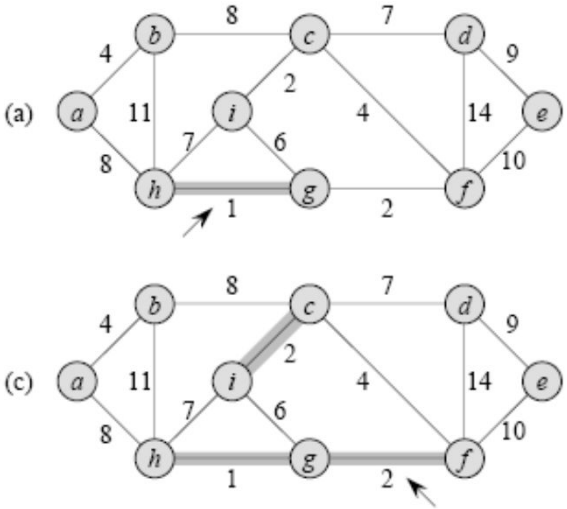
Algoritmo de Kruskal (2)

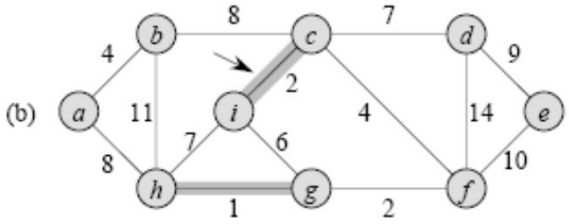
- Seleciona a aresta de menor peso que conecta duas árvores de uma floresta.
- Repita o processo até que todos os vértices estejam conectados sempre preservando a invariante de se ter uma árvore.



Algoritmo de Kruskal (3)

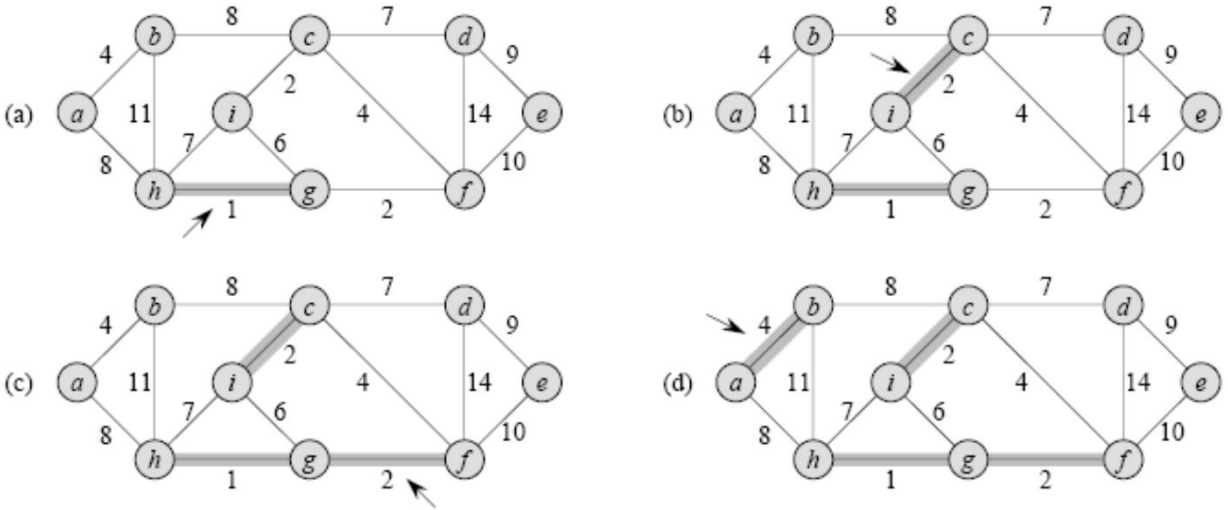
- Seleciona a aresta de menor peso que conecta duas árvores de uma floresta.
- Repita o processo até que todos os vértices estejam conectados sempre preservando a invariante de se ter uma árvore.





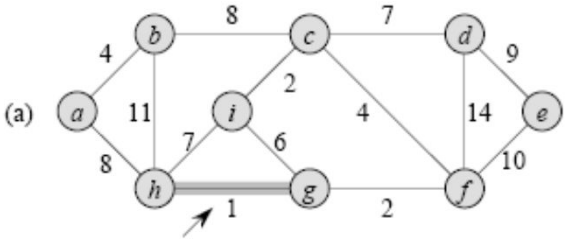
Algoritmo de Kruskal (4)

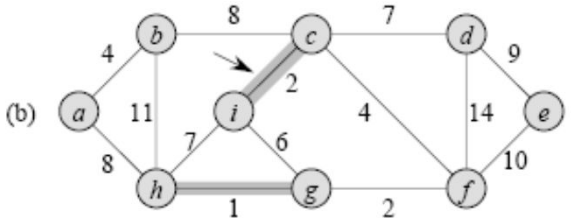
- Seleciona a aresta de menor peso que conecta duas árvores de uma floresta.
- Repita o processo até que todos os vértices estejam conectados sempre preservando a invariante de se ter uma árvore.

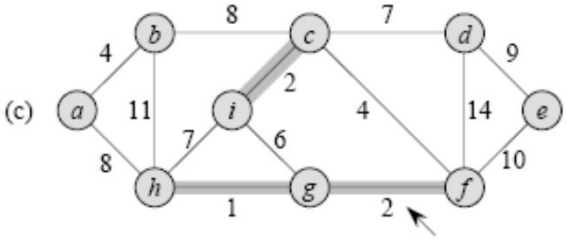


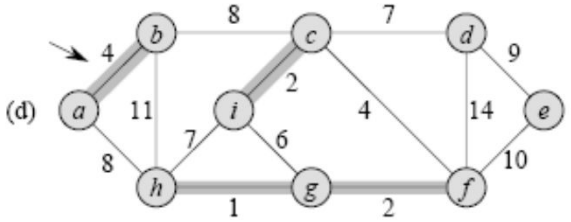
Algoritmo de Kruskal (5)

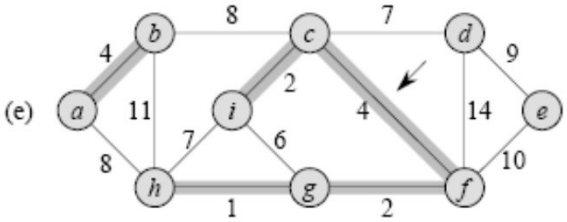
- Seleciona a aresta de menor peso que conecta duas árvores de uma floresta.
- Repita o processo até que todos os vértices estejam conectados sempre preservando a invariante de se ter uma árvore.





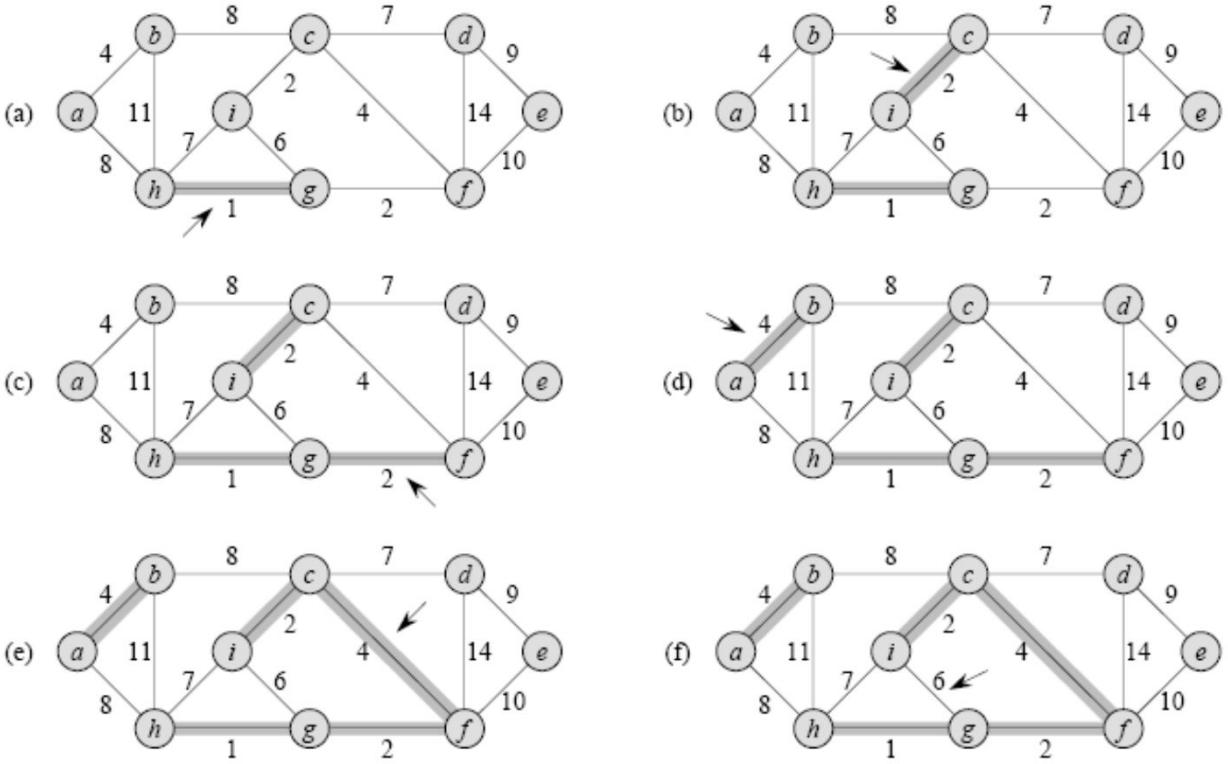






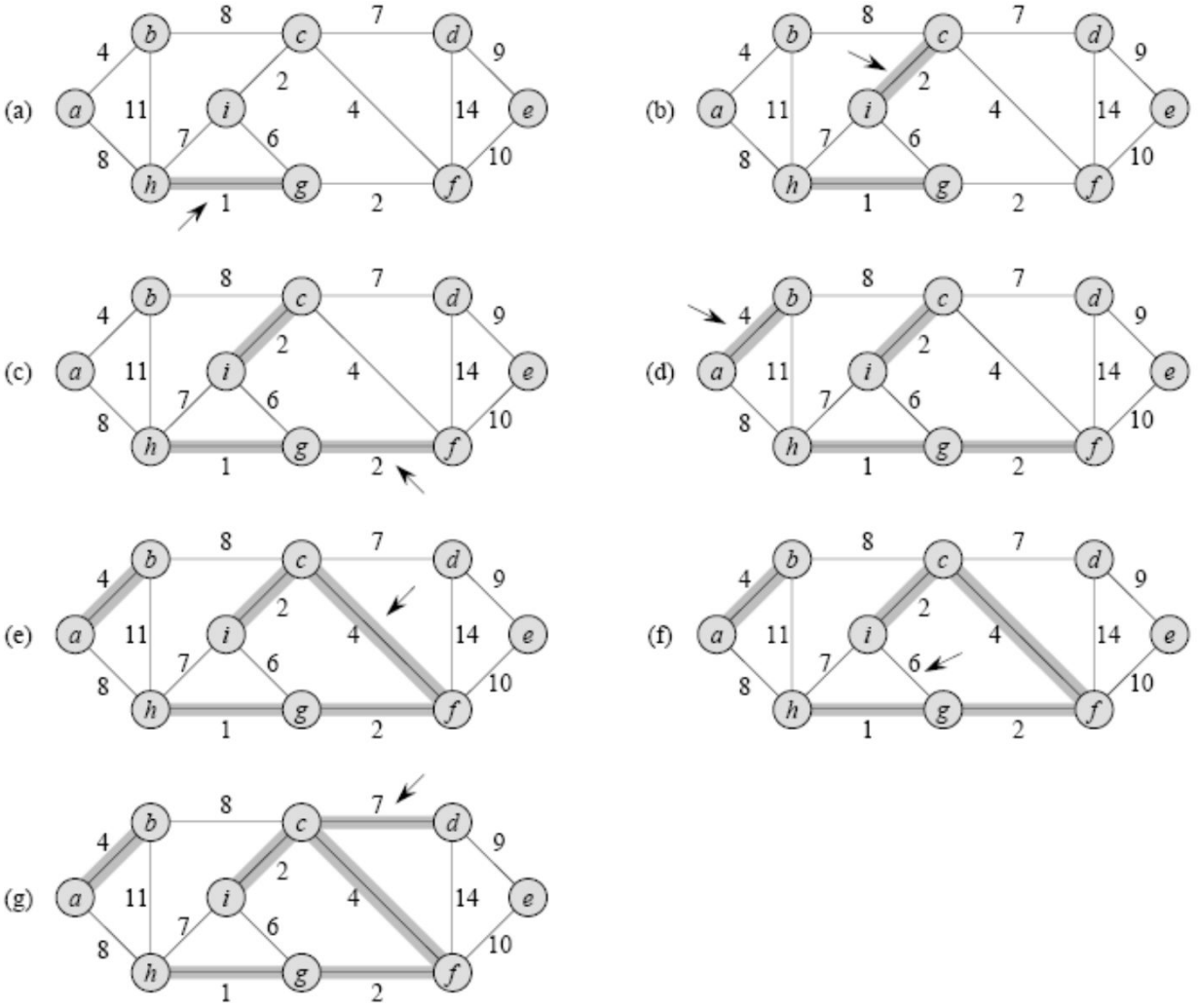
Algoritmo de Kruskal (6)

- Seleciona a aresta de menor peso que conecta duas árvores de uma floresta.
- Repita o processo até que todos os vértices estejam conectados sempre preservando a invariante de se ter uma árvore.



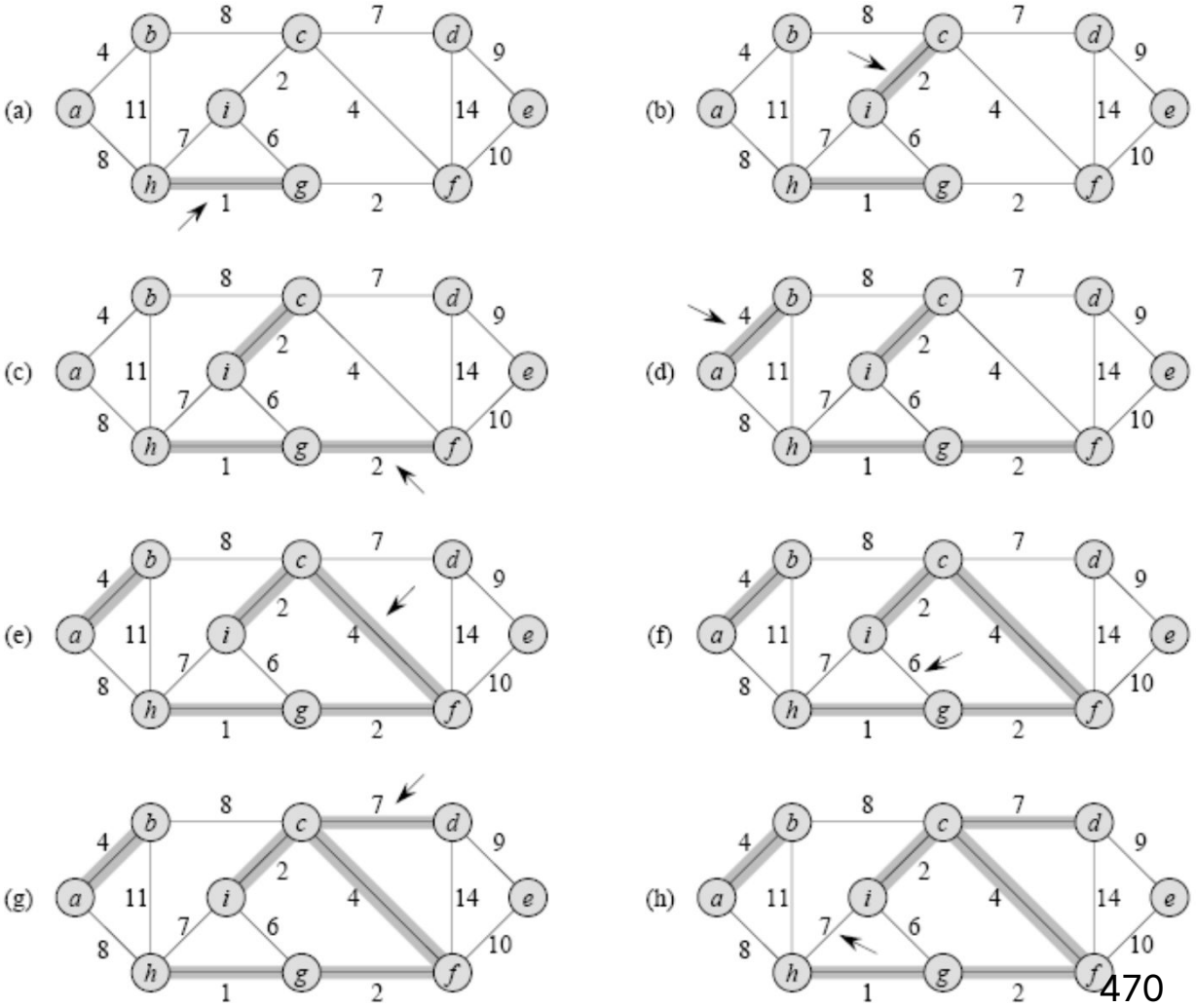
Algoritmo de Kruskal (7)

- Seleciona a aresta de menor peso que conecta duas árvores de uma floresta.
- Repita o processo até que todos os vértices estejam conectados sempre preservando a invariante de se ter uma árvore.



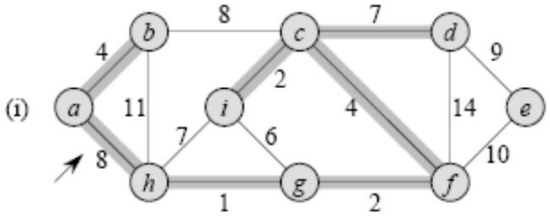
Algoritmo de Kruskal (8)

- Seleciona a aresta de menor peso que conecta duas árvores de uma floresta.
- Repita o processo até que todos os vértices estejam conectados sempre preservando a invariante de se ter uma árvore.



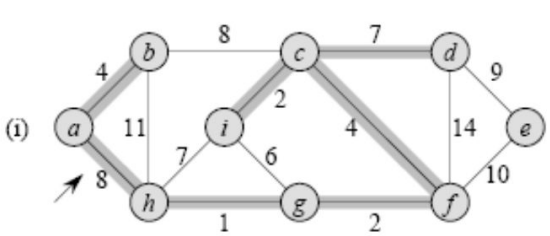
Algoritmo de Kruskal (9)

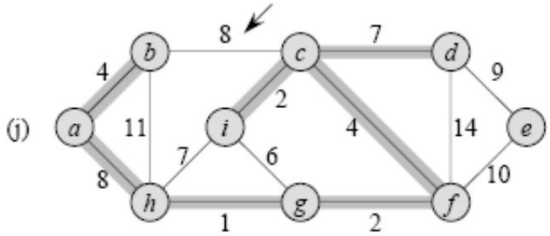
- Seleciona a aresta de menor peso que conecta duas árvores de uma floresta.
- Repita o processo até que todos os vértices estejam conectados sempre preservando a invariante de se ter uma árvore.



Algoritmo de Kruskal (10)

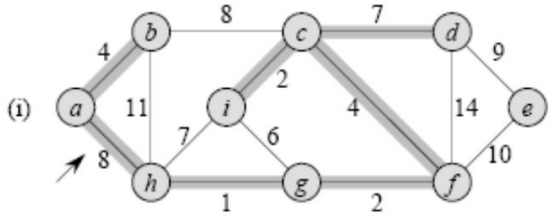
- Seleciona a aresta de menor peso que conecta duas árvores de uma floresta.
- Repita o processo até que todos os vértices estejam conectados sempre preservando a invariante de se ter uma árvore.

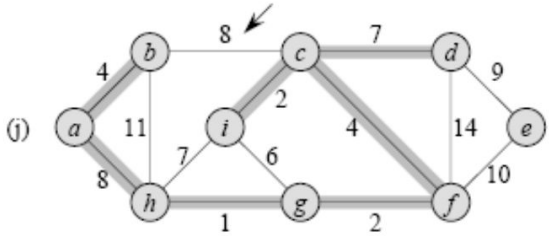


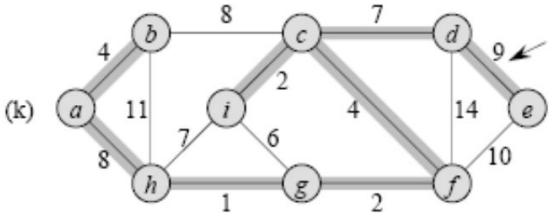


Algoritmo de Kruskal (11)

- Seleciona a aresta de menor peso que conecta duas árvores de uma floresta.
- Repita o processo até que todos os vértices estejam conectados sempre preservando a invariante de se ter uma árvore.

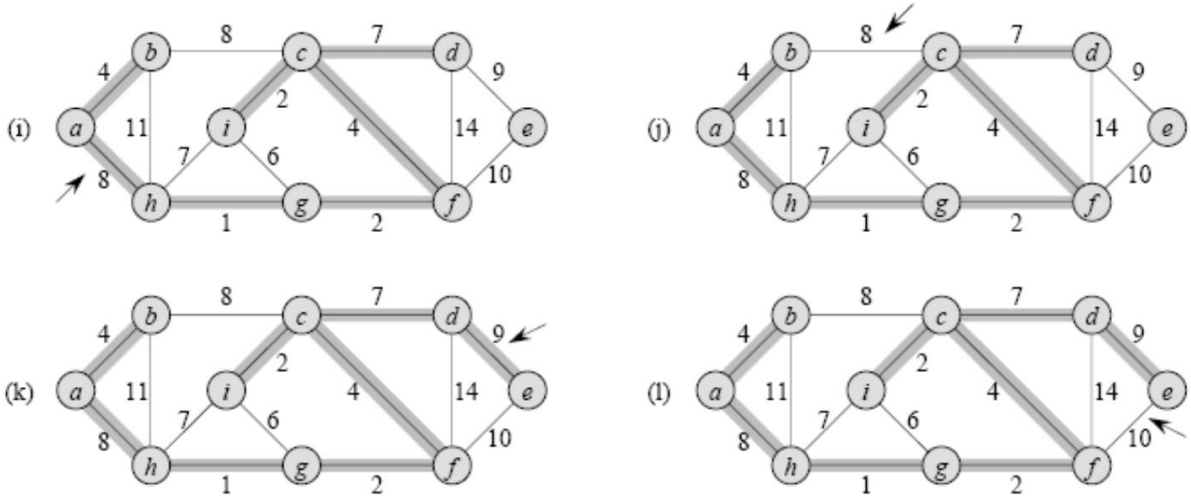






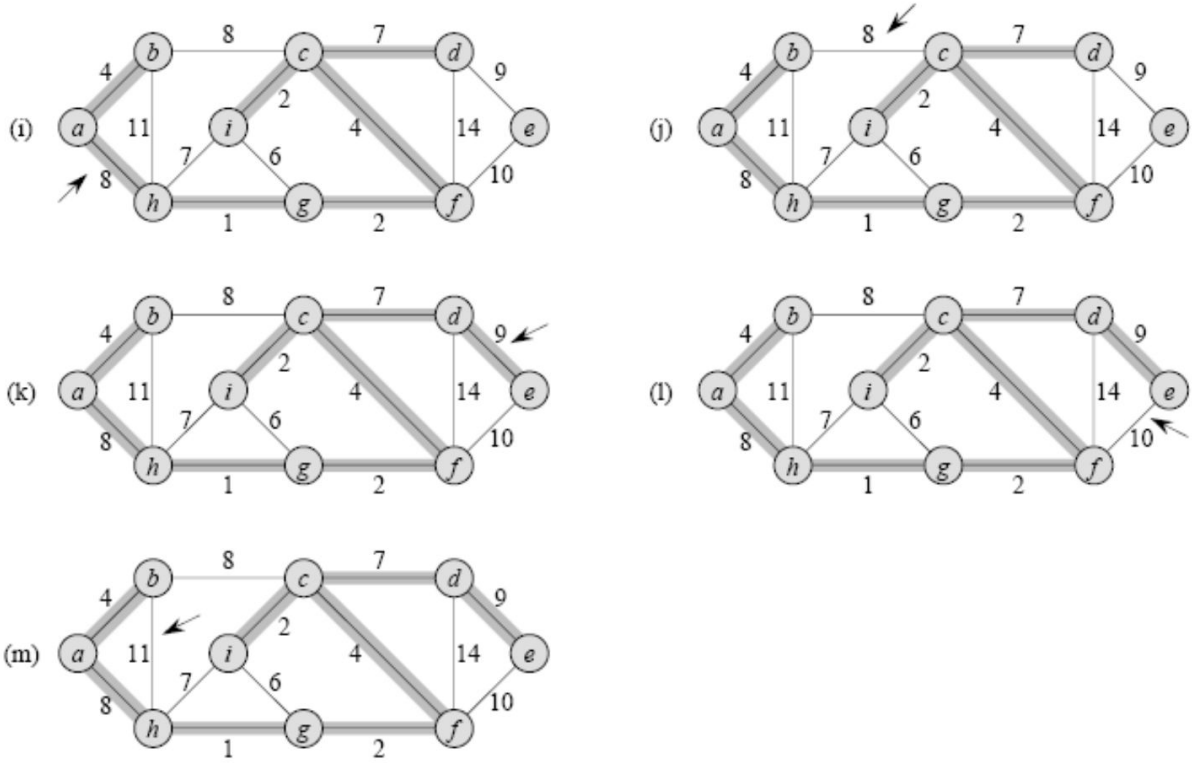
Algoritmo de Kruskal (12)

- Seleciona a aresta de menor peso que conecta duas árvores de uma floresta.
- Repita o processo até que todos os vértices estejam conectados sempre preservando a invariante de se ter uma árvore.



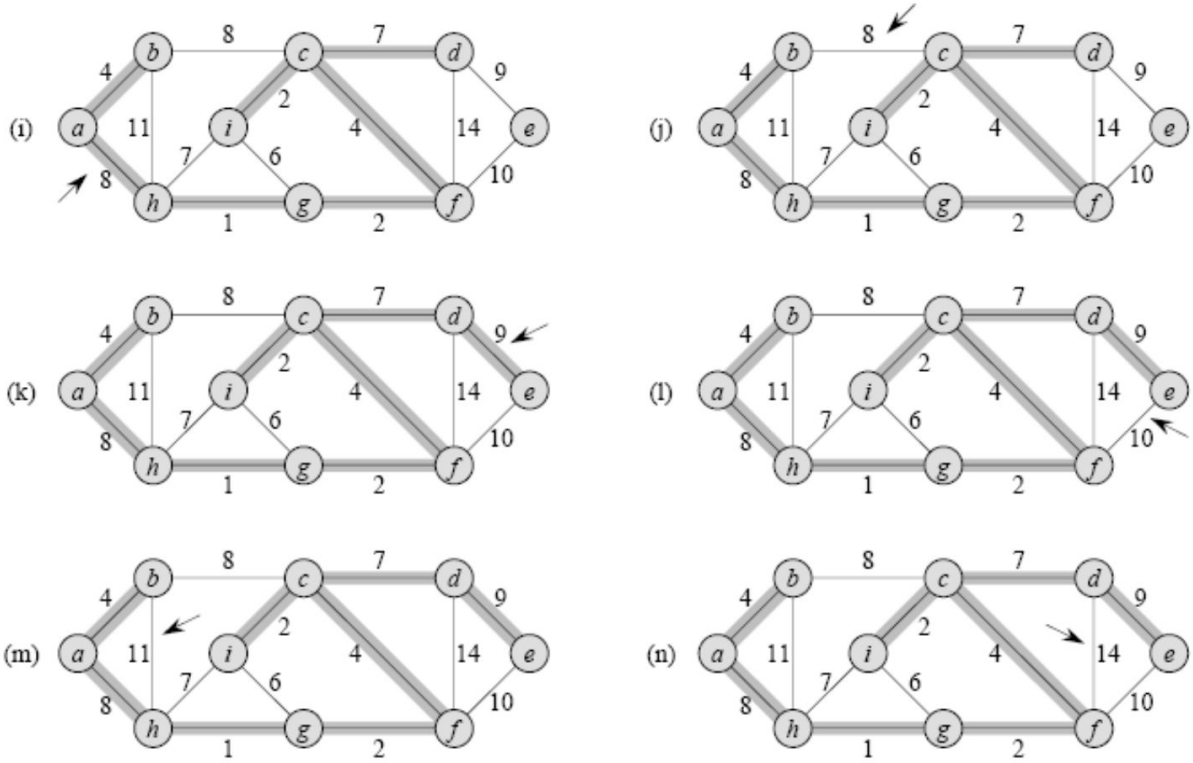
Algoritmo de Kruskal (13)

- Seleciona a aresta de menor peso que conecta duas árvores de uma floresta.
- Repita o processo até que todos os vértices estejam conectados sempre preservando a invariante de se ter uma árvore.



Algoritmo de Kruskal (14)

- Seleciona a aresta de menor peso que conecta duas árvores de uma floresta.
- Repita o processo até que todos os vértices estejam conectados sempre preservando a invariante de se ter uma árvore.



Algoritmo de Kruskal (15)

- Seleciona a aresta de menor peso que conecta duas árvores de uma floresta.
- Repita o processo até que todos os vértices estejam conectados sempre preservando a invariante de se ter uma árvore.

ALGORITHM 2 Kruskal's Algorithm.

procedure *Kruskal*(G: weighted connected undirected graph with n vertices)
T := empty graph
for $i := 1$ **to** $n - 1$
  e := any edge in G with smallest weight that does not form a simple circuit when added to T
  $T := T$ with e added
return T {T is a minimum spanning tree of G}